Lakehouse: Una nueva generación de plataformas abiertas que unifican el almacenamiento de datos y las analíticas avanzadas.

Aguilar Coronel Andy Franco[1]

Email: aaguilarc@uni.pe[1]

Abstract

Este artículo está basado en "Lakehouse: A New Generation of Open Platforms that Unify DataWarehousing and Advanced Analytics" publicado en el CIDR el 2021, se detalla el pronóstico de una nueva evolución en los almacenes de datos a la que se le llama Lakehouse, la cual se adapta mejor a las necesidades actuales como datos no estructurados, machine learning, tiempos de evaluación teniendo en cuenta la obsolescencia de datos y costos operativos, así como también los desafíos que se deben afrontar.

1 Introducción

La historia de los almacenes de datos comenzó con el objetivo de ayudar a los líderes empresariales a obtener conocimientos analíticos mediante la recopilación de datos de bases de datos operacionales en almacenes centralizados, que luego se podían utilizar para la toma de decisiones y la inteligencia empresarial (BI). En estos almacenes, los datos se escribían utilizando un esquema en el momento de la escritura, lo que garantizaba que el modelo de datos estuviera optimizado para el consumo de BI posterior. Éste es el concepto de la primera generación de plataformas de análisis de datos y tenían dos desafíos, En primer lugar combinaban la computación y el almacenamiento en un dispositivo en las instalaciones, lo que obligaba a las empresas a aprovisionar y pagar por la carga máxima de usuarios y los datos bajo gestión, lo que resultaba muy costoso a medida que los conjuntos de datos crecían. En segundo lugar, no solo los conjuntos de datos estaban creciendo rápidamente, sino que cada vez más conjuntos de datos eran completamente no estructurados, como el video, audio y documentos de texto, que los almacenes de datos no podían almacenar ni consultar en absoluto.

En la segunda generación se comenzaron a descargar todos los datos sin procesar en data lakes, usando sistemas de almacenamiento de bajo costo con una API de archivos que almacenan datos en formatos de archivos genéricos y generalmente abiertos, como Apache Parquet y ORC. Este enfoque comenzó con el movimiento de Apache Hadoop, utilizando el Sistema de Archivos Hadoop (HDFS) para un almacenamiento económico. El data lake era una arquitectura de esquema, en el momento de la lectura permitía la agilidad de almacenar cualquier dato a bajo costo, pero, por otro lado, aplazaba el problema de la calidad y administración de los datos para más adelante. En esta arquitectura, un pequeño subconjunto de datos en el data lake se le realizaba luego el ETL (ETL - Extract Transform and Load) a un almacén de datos para las aplicaciones de soporte de decisiones y BI. El uso de formatos abiertos también permitió el acceso directo a los datos del data lake por parte de una amplia gama de otros motores de análisis, como sistemas de aprendizaje automático.

A partir de 2015, los data lakes en la nube comenzaron a reemplazar al HDFS. Estos tienen una durabilidad superior (a menudo > a 10 nueves), repli-

cación geográfica y, lo más importante, un costo extremadamente bajo. El resto de la arquitectura es en gran medida la misma en la nube que en los sistemas de segunda generación. Esta arquitectura de dos niveles, data lake + almacén de datos, ahora es dominante en la industria.

En las plataformas de primera generación, a todos los datos se le realizaba ETL desde los sistemas de datos operacionales directamente hacia un almacén de datos. En las arquitecturas actuales, los datos se le hacen ETL primero en los data lakes y luego nuevamente en los almacenes de datos, lo que genera complejidad, demoras y nuevos modos de falla.

Las arquitecturas de datos actuales comúnmente sufren de cuatro problemas:

Confiabilidad. Mantener la consistencia entre el "data lake" y el almacén de datos es difícil y costoso. Se requiere ingeniería continua para realizar la extracción, transformación y carga (ETL) de datos entre los dos sistemas y hacer que estén disponibles para el soporte de decisiones de alto rendimiento y la inteligencia empresarial (BI). Cada paso de ETL también corre el riesgo de provocar fallas o introducir errores que reduzcan la calidad de los datos, por ejemplo, debido a diferencias sutiles entre los motores del "data lake" y del almacén de datos.

Obsolescencia de los datos. Los datos en el almacén de datos están obsoletos en comparación con los del data lake, y los nuevos datos a menudo tardan días en cargarse. Esto representa un retroceso en comparación con la primera generación de sistemas de análisis, donde los nuevos datos operacionales estaban disponibles de inmediato para consultas. Según una encuesta realizada por Dimensional Research y Fivetran, el 86% de los analistas utilizan datos desactualizados y el 62% informa que esperan recursos de ingeniería varias veces al mes.

Soporte limitado para análisis avanzados. Las empresas desean hacer preguntas predictivas utilizando sus datos de almacén, por ejemplo, "¿a qué clientes debo ofrecer descuentos?". A pesar de la amplia investigación sobre la convergencia de aprendizaje automático (ML) y gestión de datos, ninguno de los principales sistemas de aprendizaje automático, como TensorFlow, PyTorch y XGBoost, funciona bien en los almacenes de datos. A diferencia de las consultas de inteligencia empresarial (BI), que extraen una pequeña cantidad de datos, estos sistemas necesitan procesar conjuntos de datos grandes utilizando código no SQL complejo. Leer estos datos a través de ODBC/JDBC es ineficiente y no hay forma de acceder directamente a los formatos propietarios internos del almacén.

Costo total de propiedad. Además de pagar por la ETL continua, los usuarios pagan el doble del costo de almacenamiento por los datos copiados en un almacén de datos, y los almacenes comerciales bloquean los datos en formatos propietarios que aumentan el costo de migrar datos o cargas de trabajo a otros sistemas.

Las Soluciones que aporta el Lakehouse son las siguientes:

Gestión confiable de datos en los data lakes. Un Lakehouse debe ser capaz de almacenar datos en bruto, de manera similar a los data lakes actuales, al mismo tiempo que admite procesos de ETL/ELT que curan estos datos para mejorar su calidad para el análisis.

Soporte para aprendizaje automático y ciencia de datos. El soporte de los sistemas de aprendizaje automático para la lectura directa de formatos de data lake ya los coloca en una buena posición para acceder eficientemente a una Lakehouse.

Rendimiento de SQL. Las Lakehouses deberán proporcionar un rendimiento de SQL de vanguardia sobre los masivos conjuntos de datos Parquet/ORC que se han acumulado en la última década. Estos formatos que son clásicos aceptan SQL, incluso los datos propietarios.

2 Desafíos del DataWarehousing

En primer lugar, el principal problema reportado por los usuarios de datos empresariales hoy en día suele ser la calidad y confiabilidad de los datos. Implementar canalizaciones de datos correctas es muy complicado, pero las arquitecturas de datos actuales de dos niveles que tienen un data lake y un almacén por separado, agregan una complejidad adicional que empeora este problema.

En segundo lugar, cada vez más aplicaciones empresariales requieren datos actualizados, pero las arquitecturas actuales aumentan la falta de actualidad de los datos al tener un área de preparación separada para los datos entrantes antes del almacén y utilizar

trabajos periódicos de ETL/ELT para cargarlos. Las aplicaciones empresariales como sistemas de soporte al cliente y motores de recomendación no pueden funcionar bien con datos desactualizados, e incluso los analistas humanos que consultan almacenes informan con datos desactualizados, este es un problema importante.

En tercer lugar, una gran cantidad de datos ahora son no estructurados en muchas industrias, ya que las organizaciones recopilan imágenes, datos de sensores, documentos, etc. Las organizaciones necesitan sistemas fáciles de usar para gestionar estos datos, pero los almacenes de datos SQL y su API no están diseñadas para este objetivo.

Por último, la mayoría de las organizaciones están implementando aplicaciones de aprendizaje automático y ciencia de datos, pero no son adecuadas para los almacenes de datos y los data lakes.

3 La arquitectura de la Lakehouse

Definimos una Lakehouse como un sistema de gestión de datos basado en almacenamiento de bajo costo y de acceso directo que también proporciona características tradicionales de administración y rendimiento de un sistema de gestión de bases de datos analíticas, como transacciones ACID, versionado de datos, auditoría, indexación, almacenamiento en caché y optimización de consultas. Por lo tanto, las Lakehouses combinan los principales beneficios de los data lakes y los data warehouses: almacenamiento de bajo costo en un formato abierto accesible por una variedad de sistemas proveniente del primero, y características de gestión y optimización potentes provenientes del último. La pregunta clave es si se pueden combinar estos beneficios de manera efectiva, el soporte de las Lakehouses también implica que se renuncie a ciertos aspectos de la independencia de datos, que ha sido un elemento fundamental en el diseño de los sistemas de bases de datos relacionales.

Destacamos que las Lakehouses se adaptan especialmente bien a entornos en la nube con almacenamiento y procesamiento separados: diferentes aplicaciones de cómputo pueden ejecutarse bajo demanda en nodos de cómputo completamente separados (por ejemplo, un clúster de GPU para ML) mientras acceden directamente a los mismos datos de almacenamiento. Sin embargo, también se podría implementar una Lakehouse sobre un sistema de almacenamiento local como HDFS.

3.1 Implementación de un sistema Lakehouse

La primera idea clave que proponemos para implementar una Lakehouse es que el sistema almacene los datos en un almacenamiento de objetos de bajo costo (por ejemplo, Amazon S3) utilizando un formato de archivo estándar como Apache Parquet, pero implemente una capa de metadatos transaccional en la parte superior del almacenamiento de objetos que define qué objetos forman parte de una versión de tabla. Esto permite que el sistema implemente características de gestión como transacciones ACID o versionado dentro de la capa de metadatos, al tiempo que mantiene la mayor parte de los datos en el almacenamiento de objetos de bajo costo y permite a los clientes leer directamente objetos de este almacenamiento utilizando un formato de archivo estándar en la mayoría de los casos.

Aunque una capa de metadatos agrega capacidades de gestión, no es suficiente para lograr un buen rendimiento de SQL. Los data warehouses utilizan varias técnicas para obtener un rendimiento de vanguardia, como almacenar datos activos en dispositivos rápidos como SSD, mantener estadísticas, construir métodos de acceso eficientes como índices y co-optimizar el formato de datos y el motor de cálculo. En una Lakehouse basada en formatos de almacenamiento existentes, no es posible cambiar el formato, pero mostramos que es posible implementar otras optimizaciones que dejan los archivos de datos sin cambios, incluyendo el almacenamiento en caché, estructuras de datos auxiliares como índices y estadísticas, y optimizaciones de disposición de datos.

Finalmente, las Lakehouses pueden acelerar las cargas de trabajo de análisis avanzado y brindarles mejores características de gestión de datos gracias al desarrollo de las APIs declarativas de DataFrame. Muchas bibliotecas de ML, como TensorFlow y Spark MLlib, ya pueden leer formatos de archivos de data lake. Por lo tanto, la forma más sencilla de integrarlas con una Lakehouse sería consultar la capa de metadatos para determinar qué archivos forman parte actualmente de una tabla y simplemente pasar esos archivos a la biblioteca de ML. También

la mayoría de estos sistemas admiten una API de DataFrame para la preparación de datos que crea más oportunidades de optimización.

3.2 Capas de metadatos para la gestión de datos

El primer componente que permitirá la adaptación a Lakehouses son las capas de metadatos sobre el almacenamiento de data lake que pueden elevar su nivel de abstracción para implementar transacciones ACID y otras características de gestión. Las organizaciones han diseñado capas de gestión de datos más ricas sobre estos sistemas pasando de S3 o HDFS que no son atómicas a Apache Hive ACID que realiza un seguimiento de qué archivos de datos forman parte de una tabla de Hive en una versión de tabla dada. En los últimos años, nuevos sistemas han proporcionado aún más capacidades y mejorado la escalabilidad. En 2016, Databricks comenzó a desarrollar Delta Lake, que almacena la información sobre qué objetos forman parte de una tabla en el propio "data lake" como un registro de transacciones lo que le permite escalar a miles de millones de objetos por tabla, otro es Apache Hudi, que comenzó en Uber, es otro sistema en esta área enfocado en simplificar la ingestión de transmisión en los data lakes, aunque no admite escritura concurrente.

La experiencia con estos sistemas ha demostrado que generalmente proporcionan un rendimiento similar o mejor que los data lakes sin procesar, al tiempo que agregan características de gestión altamente útiles como transacciones, clonación sin copia y acceso a versiones anteriores de una tabla. Además, son fáciles de adoptar para las organizaciones que ya tienen un data lake. Además, las capas de metadatos son un lugar natural para implementar características de cumplimiento de la calidad de los datos.

3.3 Rendimiento de SQL en un Lakehouse

Una pregunta importante es como darle un buen rendimiento SQL si se deja atrás el diseño tradicional. La respuesta depende de varios factores, como los recursos de hardware disponibles y si podemos cambiar el formato de almacenamiento de objetos de datos en lugar de utilizar estándares existentes.

Proponemos varias técnicas para implementar optimizaciones de rendimiento de SQL en un Lakehouse independientes del formato de datos elegido, por lo que se pueden aplicar tanto con formatos existentes como futuros.

Caching: Cuando se utiliza una capa de metadatos transaccional como Delta Lake, es seguro para un sistema Lakehouse almacenar en caché archivos del objeto store en la nube en dispositivos de almacenamiento más rápidos, como SSD y RAM en los nodos de procesamiento. Las transacciones en ejecución pueden determinar fácilmente cuándo los archivos en caché siguen siendo válidos para su lectura.

Datos auxiliares: Aunque un Lakehouse necesita exponer el formato de almacenamiento de la tabla base para E/S directa, puede mantener otros datos que ayuden a optimizar las consultas en archivos auxiliares sobre los cuales tiene control total.

Distribución de datos: La distribución de datos juega un papel importante en el rendimiento del acceso. Incluso cuando fijamos un formato de almacenamiento, hay múltiples decisiones de distribución que pueden ser optimizadas por el sistema Lakehouse, la más obvia es el orden de los registros: qué registros se agrupan juntos y, por lo tanto, son más fáciles de leer en conjunto, también se pueden imaginar nuevos formatos que admitan la colocación de columnas en diferentes órdenes dentro de cada archivo de datos, la elección de estrategias de compresión de manera diferente para varios grupos de registros, u otras estrategias

Resultados de rendimiento. En Databricks, combinamos estas tres optimizaciones del sistema Lakehouse con un nuevo motor de ejecución en C++ para Apache Spark llamado Delta Engine, Delta Engine proporciona un rendimiento comparable o mejor que estos sistemas a un costo más bajo.

Direcciones futuras y diseños alternativos. El diseño de sistemas Lakehouse de alto rendimiento, pero directamente accesibles es un área rica para trabajos futuros. Una dirección clara que aún no hemos explorado es el diseño de nuevos formatos de almacenamiento de data lake que funcionen mejor en este caso de uso, por ejemplo, formatos que brinden más flexibilidad al sistema Lakehouse para implementar optimizaciones de distribución de datos o índices, o que simplemente se adapten mejor al hardware moderno.

3.4 Acceso eficiente para análisis avanzados

Las bibliotecas de análisis avanzado suelen estar escritas utilizando código imperativo que no se puede ejecutar como SQL, pero aún así necesitan acceder a grandes cantidades de datos. Existe una interesante pregunta de investigación sobre cómo diseñar las capas de acceso a datos en estas bibliotecas para maximizar la flexibilidad del código. Un enfoque con el que hemos tenido éxito es ofrecer una versión declarativa de las API de DataFrame utilizadas en estas bibliotecas, que mapea las operaciones de preparación de datos en planes de consulta de Spark SQL y puede beneficiarse de las optimizaciones en Delta Lake y Delta Engine. En nuestra implementación de la fuente de datos de Delta Lake, aprovechamos las optimizaciones de almacenamiento en caché, omisión de datos y distribución de datos. Las API de aprendizaje automático están evolucionando rápidamente y también existen otras API de acceso a datos, como tf.data de TensorFlow, que no intentan empujar la semántica de consulta hacia el sistema de almacenamiento subyacente.

Direcciones futuras y diseños alternativos. trabajos recientes han propuesto marcos de trabajo de "ML factorizado" que empujan la lógica de ML hacia las uniones de SQL y otras optimizaciones de consulta que se pueden aplicar a algoritmos de ML implementados en SQL. Por último, todavía necesitamos interfaces estándar que permitan a los científicos de datos aprovechar al máximo las potentes capacidades de gestión de datos en Lakehouses (o incluso almacenes de datos).

4 Preguntas de investigación e implicaciones

¿Existen otras formas de lograr los objetivos de un Lakehouse? Se pueden imaginar otros medios para lograr los objetivos principales de un Lakehouse, como construir una capa de servicio de lectura masivamente paralela para un almacén de datos que pueda admitir lecturas paralelas de cargas de trabajo de análisis avanzado. Sin embargo, creemos que tal infraestructura será significativamente más costosa de ejecutar, más difícil de gestionar y probablemente menos eficiente que brindar a las cargas de trabajo acceso directo al almacenamiento de objetos. No hemos visto una implementación generalizada de sistemas que agreguen este tipo de capa de servicio.

¿Cuáles son los formatos de almacenamiento y las API de acceso adecuados? La interfaz de acceso a un Lakehouse incluye el formato de almacenamiento en bruto, las bibliotecas de cliente para leer directamente este formato (por ejemplo, al leer en TensorFlow) y una interfaz SQL de alto nivel. Hay muchas formas diferentes de ubicar funcionalidades avanzadas en estas capas, como esquemas de almacenamiento que proporcionan más flexibilidad al sistema al solicitar a los lectores que realicen una lógica de decodificación más sofisticada y "programable".

¿Cómo afecta el Lakehouse a otras investigaciones y tendencias en la gestión de datos? La prevalencia de los data lakes y el uso cada vez mayor de interfaces de gestión avanzadas sobre ellos, ya sea en forma de capas de metadatos o en el diseño completo del Lakehouse, tiene implicaciones en varias otras áreas de investigación en gestión de datos.

Los polystores fueron diseñados para resolver el difícil problema de consultar datos en diferentes motores de almacenamiento. Muchas consultas de polystore podrían responderse ejecutándose directamente en el objeto de almacenamiento en la nube, incluso si los archivos de datos subyacentes son parte de implementaciones lógicamente separadas del Lakehouse.

Las herramientas de integración y limpieza de datos también se pueden diseñar para ejecutarse en su lugar sobre un Lakehouse con acceso paralelo rápido a todos los datos, lo que puede permitir nuevos algoritmos.

La gestión de datos para ML también podría volverse más sencilla y potente si se implementa sobre un Lakehouse. En la actualidad, las organizaciones están construyendo una amplia gama de sistemas de versionado de datos específicos de ML y "almacenamiento de características" que vuelven a implementar la funcionalidad estándar de los sistemas de gestión de bases de datos.

Los diseños de DBMS nativos de la nube, como los motores sin servidor, deberán integrarse con capas de gestión de metadatos más ricas, como Delta Lake, en lugar de simplemente escanear archivos sin procesar en un data lake.

5 Conclusión

Se ha argumentado que una arquitectura de plataforma de datos unificada que implementa la funcionalidad de un almacén de datos sobre formatos de archivos abiertos en un data lake puede proporcionar un rendimiento competitivo con los sistemas de almacén de datos actuales y ayudar a abordar muchos de los desafíos que enfrentan los usuarios de almacenes de datos. Aunque limitar la capa de almacenamiento de un almacén de datos a archivos abiertos y directamente accesibles en un formato estándar puede parecer una limitación significativa al principio, optimizaciones como el almacenamiento en caché para datos frecuentemente accedidos y la optimización de la disposición de datos para datos menos frecuentes pueden permitir que los sistemas Lakehouse logren un rendimiento competitivo. Creemos que es probable que la industria converja hacia diseños de Lakehouse dada la gran cantidad de datos que ya se encuentran en los data lakes y la oportunidad de simplificar en gran medida las arquitecturas de datos empresariales.